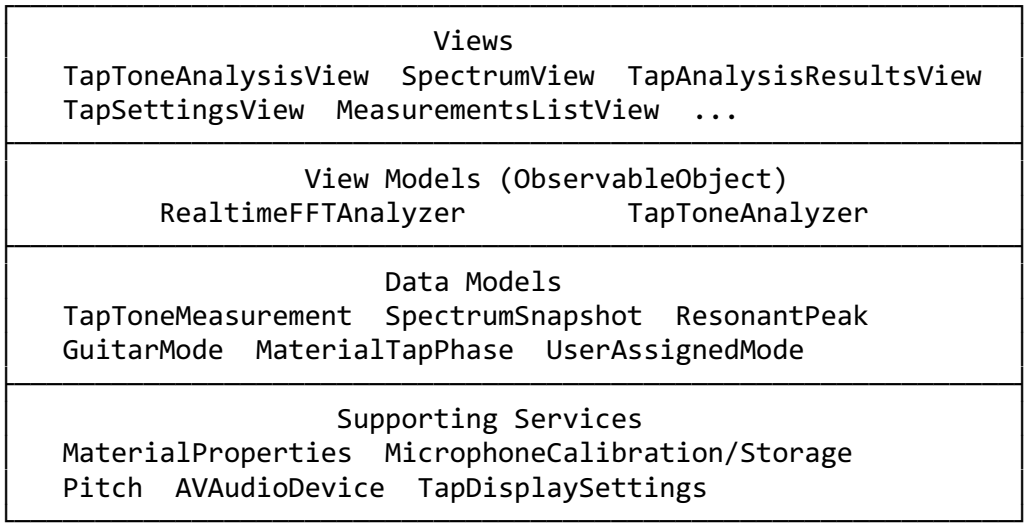
GuitarTap Architecture

GuitarTap is a cross-platform (macOS + iOS) luthier tool for tap-tone analysis. Users tap a guitar top, back, plate, or brace and the app captures the acoustic response via microphone, performs real-time FFT analysis, identifies resonant modes, and calculates material properties.

Layered Overview



The pattern is MVVM. Both view models are `ObservableObjects` instantiated as `@StateObject` in `TapToneAnalysisView` and passed down the tree as `@ObservedObject`. `TapToneAnalyzer` subscribes directly to `RealtimeFFTAnalyzer`'s published values — it does not own its own audio session.

View Models

RealtimeFFTAnalyzer

Files: `RealtimeFFTAnalyzer.swift` + `+DeviceManagement`, `+EngineControl`, `+FFTProcessing`

Owns the `AVAudioEngine` and `vDSP` DFT setup. Responsible for everything below the spectrum data.

Publishes:

Property	Rate	Description
`frequencies[]`, `magnitudes[]`	~1–2 Hz	Full FFT spectrum (65,536-point, 48 kHz target)
`peakMagnitude`, `peakFrequency`	~1–2 Hz	Dominant bin in the current spectrum
`inputLevelDB`, `recentPeakLevelDB`	~10 Hz	Fast signal level for decay tracking and tap detection

Two FFT modes: - **Continuous** (rectangular window) — always running; drives live spectrum display and guitar-mode tap detection. - **Gated one-shot** (Hann window) — triggered by `TapToneAnalyzer` for plate/brace captures; uses a temporary DFT setup so it doesn't interfere with the continuous path.

Device management is platform-specific: CoreAudio `AudioDeviceID` on macOS, `AVAudioSessionPortDescription` on iOS. When the selected device changes, calibration data is automatically loaded via `CalibrationStorage`.

TapToneAnalyzer

Files: `TapToneAnalyzer.swift` + 9 extension files

The central coordinator. Subscribes to `RealtimeFFTAnalyzer` and implements the full tap-detection and analysis pipeline. Each extension owns one well-defined responsibility:

Extension	Responsibility
`+TapDetection`	Rising-edge hysteresis detector. Guitar mode: absolute `peakMagnitude` threshold. Plate/brace: `noiseFloor EMA + headroom` (relative). Warm-up suppression for first 0.5 s.
`+PeakAnalysis`	Two-pass peak finder: pass 1 guarantees one slot per known `GuitarMode` frequency range, pass 2 fills remaining slots by magnitude. Parabolic sub-bin interpolation. Q-factor via −3 dB bandwidth method.
`+SpectrumCapture`	Gated PCM capture seeded from pre-roll ring buffer. HPS + Q-factor filtering selects dominant material resonance (rejects Q < 3). Frequency-domain power averaging across multi-tap captures.

Extension	Responsibility
`+DecayTracking`	Opens a 3-second ring-out window after each tap. Measures time from peak level to a threshold crossing (`recentPeakLevelDB` fast path).
`+MeasurementManagement`	JSON persistence to `Documents/saved_measurements.json`. Import/export. PNG sidecar chart images.
`+ModeOverrideManagement`	Thread-safe bulk apply/reset of user-supplied mode labels.
`+AnnotationManagement`	Per-peak `[UUID: CGPoint]` drag offsets for chart labels.
`+Control`	Lifecycle (`start/stop/reset`), tap sequence state machine (`startTapSequence/pause/resume/cancel`).
`+AnalysisHelpers`	Tap-tone ratio (`f_Top / f_Air`, ideal ≈ 2.0), measurement-to-measurement frequency delta comparison.

Key published state:

Property	Description
`currentPeaks`	Detected `ResonantPeak` array, updated at FFT rate
`identifiedModes`	Mode classifications for current peaks
`isDetecting`, `tapProgress`	Detection active flag and multi-tap progress (0–1)
`isSpectrumFrozen`, `frozenFrequencies/Magnitudes`	Frozen display after a completed tap sequence
`currentDecayTime`	Ring-out measurement in seconds
`materialTapPhase`	Current phase in plate/brace multi-phase sequence
`savedMeasurements`	Persisted `[TapToneMeasurement]`

Data Models

TapToneMeasurement

Serializable record of one complete session. Captures peaks, decay time, tap location, notes, per-phase spectrum snapshots (guitar uses a single `spectrumSnapshot`; plate/brace

add `longitudinalSnapshot`, `crossSnapshot`, `flcSnapshot`), display state (annotation offsets, selected peaks, visibility mode, mode overrides), analysis settings (thresholds, tap count, peak count), and microphone provenance (name, UID, calibration name).

SpectrumSnapshot

Immutable frozen spectrum paired with its display axis settings (`minFreq`, `maxFreq`, `minDB`, `maxDB`, `isLogarithmic`) and the analysis settings active at capture time (guitar type, measurement type, plate/brace dimensions, stiffness parameters).

ResonantPeak

`Identifiable` + `Codable`. Fields: `frequency`, `magnitude` (dBFS), `quality` (Q-factor), `bandwidth`, pitch information (`pitchNote`, `pitchCents`, `pitchFrequency`), `timestamp`.

GuitarMode

Enum with 7 active cases (`air`, `top`, `back`, `dipole`, `ringMode`, `upperModes`, `unknown`) plus 4 legacy names for backward compatibility. `classify(frequency:guitarType:)` maps a frequency to a mode using per-guitar-type frequency ranges. Each mode has both a detection range and a narrower `idealRange`.

MaterialTapPhase

State machine for plate/brace multi-phase measurement:

```
notStarted → capturingLongitudinal → waitingForCrossTap → capturingCross
           → waitingForFlcTap → capturingFlc → complete
```

Each case exposes `instruction` and `shortStatus` strings for UI display.

Supporting Services

MaterialProperties — `PlateProperties` and `BraceProperties` compute Young's moduli from the Euler-Bernoulli beam formula and the Gore plate formula. `WoodQuality` rates results against species-specific thresholds (Sitka, Engelmann, European Spruce, Cedar). `goreTargetThickness()` calculates optimal plate thickness from Gore Equations 4.5–7.

MicrophoneCalibration / CalibrationStorage — Parses UMIK-1 / REW `.cal` files into frequency-correction point arrays. `correction(at:)` uses linear interpolation with flat extrapolation at the extremes. `CalibrationStorage` is a singleton that persists calibrations and device-UID → calibration mappings in UserDefaults. Calibration corrections are pre-computed and applied at FFT time.

Pitch — Equal-temperament calculator with configurable A4 reference (default 440 Hz). Anchored at C0 (`f_A4 × 2^(−4.75)`).

AVAudioDevice — Platform-agnostic device wrapper whose identity is based on a persistent `uid` string, not a transient runtime ID.

TapDisplaySettings — Singleton backed by UserDefaults. Global display state: frequency axis range, dB axis range, logarithmic toggle, peak threshold, unknown-mode visibility, max peak count.

Reactive Data Flow

```
AVAudioEngine install tap (audio processing queue)
        │
        ▼
RealtimeFFTAnalyzer
        │
        ├─ magnitudes / frequencies @ ~1–2 Hz ──► TapToneAnalyzer.analyzeMagn
itudes()
        │                                                ├─ findPeaks() → curr
entPeaks
        │                                                └─ detectTap() [guita
r path]
        │
        └─ inputLevelDB @ ~10 Hz ──────────────────► TapToneAnalyzer.trackDecayF
ast()
                                                         TapToneAnalyzer.detectTap()
 [plate/brace]
```

Threading rules: - Audio capture and FFT computation run on `audioProcessingQueue` (serial, `.userInteractive`). - Raw buffer access is synchronized via `bufferAccessQueue`. - All `@Published` property updates are dispatched to the main thread before assignment.

View Hierarchy & Navigation

`TapToneAnalysisView` is the root view. It owns both view models and provides platform-adaptive layouts.

Layout by platform

Platform	Layout
macOS	`HSplitView`: spectrum on left, collapsible right panel. Results open in a free-floating `NSWindow` via `ResultsWindowPresenter`.
iOS iPad	Vertical stack. Results/metrics panel appears as an inline side panel.

Platform	Layout
iOS iPhone (portrait)	Vertical stack. All secondary panels presented as sheets.
iOS iPhone (landscape)	`HStack`: controls panel on left, spectrum on right.

View tree

```
TapToneAnalysisView (root)
├── SpectrumView                    — interactive Swift Charts spectrum
│   ├── PeakAnnotations             — draggable mode/frequency labels
│   └── SpectrumScrollWheel         — macOS scroll-wheel event bridge
├── TapAnalysisResultsView          — peaks table, material properties, expor
t
├── TapSettingsView                 — detection params, display ranges, dimen
sions, device
├── MeasurementsListView
│   └── MeasurementDetailView       — playback of saved measurements
├── FFTAnalysisMetricsView          — FFT diagnostics panel
├── SaveMeasurementSheet            — captures tap location and notes
├── HelpView
└── ExportView
```

`ExportableSpectrumChart` is a gesture-free clone of `SpectrumView` used exclusively as an `ImageRenderer` target for PNG/PDF export — it never appears on screen.

Persistence & Export

Data	Location	Format
Measurements	`Documents/saved_measurements.json`	JSON array, pretty-printed, sorted keys
Chart images	Same directory, sidecar files	PNG, referenced by filename in JSON
Calibrations	UserDefaults (`CalibrationStorage`)	Codable structs
Device→calibration map	UserDefaults	`[uid: calibrationID]`
Display settings	UserDefaults (`TapDisplaySettings`)	Individual keys

Export formats available to the user: JSON bundle (measurement + PNG), PNG spectrum only, PDF report (via `PDFReportGenerator` combining spectrum image + computed data table).

File Map

```
GuitarTap/
├── GuitarTapApp.swift
├── Models/
│   ├── RealtimeFFTAnalyzer.swift
│   ├── RealtimeFFTAnalyzer+DeviceManagement.swift
│   ├── RealtimeFFTAnalyzer+EngineControl.swift
│   ├── RealtimeFFTAnalyzer+FFTProcessing.swift
│   ├── TapToneAnalyzer.swift
│   ├── TapToneAnalyzer+AnalysisHelpers.swift
│   ├── TapToneAnalyzer+AnnotationManagement.swift
│   ├── TapToneAnalyzer+Control.swift
│   ├── TapToneAnalyzer+DecayTracking.swift
│   ├── TapToneAnalyzer+MeasurementManagement.swift
│   ├── TapToneAnalyzer+ModeOverrideManagement.swift
│   ├── TapToneAnalyzer+PeakAnalysis.swift
│   ├── TapToneAnalyzer+SpectrumCapture.swift
│   ├── TapToneAnalyzer+TapDetection.swift
│   ├── TapToneMeasurement.swift
│   ├── SpectrumSnapshot.swift
│   ├── ResonantPeak.swift
│   ├── GuitarMode.swift
│   ├── MaterialProperties.swift
│   ├── MaterialTapPhase.swift
│   ├── MicrophoneCalibration.swift
│   ├── Pitch.swift
│   ├── UserAssignedMode.swift
│   └── AVAudioDevice.swift
└── Views/
    ├── TapToneAnalysisView.swift
    ├── TapToneAnalysisView+Actions.swift
    ├── TapToneAnalysisView+Controls.swift
    ├── TapToneAnalysisView+Export.swift
    ├── TapToneAnalysisView+Layouts.swift
    ├── TapToneAnalysisView+SpectrumViews.swift
    ├── SpectrumView.swift
    ├── SpectrumView+ChartContent.swift
    ├── SpectrumView+GestureHandlers.swift
    ├── SpectrumView+SnapInterpolation.swift
    ├── TapAnalysisResultsView.swift
    ├── FFTAnalysisMetricsView.swift
    ├── HelpView.swift
    ├── PeakAnnotations.swift
    ├── SaveMeasurementSheet.swift
    ├── SpectrumScrollWheel.swift
    ├── ExportableSpectrumChart.swift
    ├── Measurements/
    │   ├── MeasurementsListView.swift
    │   ├── MeasurementDetailView.swift
```

```
│   ├── MeasurementRowView.swift
│   └── ExportView.swift
├── Shared/
│   ├── CombinedPeakModeRowView.swift
│   ├── CompactFFTMetricsOverlay.swift
│   ├── EmptyStateView.swift
│   └── LoadingOverlay.swift
└── Utilities/
    ├── TapDisplaySettings.swift
    ├── TapSettingsView.swift
    ├── TapSettingsView+Actions.swift
    ├── TapSettingsView+LayoutHelpers.swift
    ├── TapSettingsView+Sections.swift
    ├── PDFReportGenerator.swift
    ├── MeasurementFileExporter.swift
    ├── ResultsWindowPresenter.swift
    ├── AxixTickGenerator.swift
    ├── PlatformAdapters.swift
    └── Extensions.swift
```

Last updated: March 2026